



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



TFG del Grado en Ingeniería
Informática

**VRLearnTobot Laboratorio
Virtual de programación y
robótica**



Presentado por Luis Ignacio de Luna Gómez
en Universidad de Burgos — 4 de marzo
de 2025

Tutor: Carlos López Nozal



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. nombre tutor, profesor del departamento de nombre departamento, área de nombre área.

Expone:

Que el alumno D. Luis Ignacio de Luna Gómez, con DNI dni, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado título de TFG.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 4 de marzo de 2025

Resumen

En este primer apartado se hace una **breve** presentación del tema que se aborda en el proyecto.

Descriptores

Palabras separadas por comas que identifiquen el contenido del proyecto Ej: servidor web, buscador de vuelos, android ...

Abstract

A **brief** presentation of the topic addressed in the project.

Keywords

keywords separated by commas.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
2. Objetivos del proyecto	5
2.1. Introducción	5
2.2. Objetivos funcionales	6
2.3. Objetivos no funcionales	7
3. Conceptos teóricos	9
3.1. Paleta de colores	9
3.2. Implementación de UBlockly en Unity	10
3.3. Bloques	11
3.4. Conexiones	12
3.5. Entradas o Inputs	13
3.6. Campos o Fields	14
3.7. Variables o VariableMap	14
3.8. Representación de bloques en Unity	14
3.9. Serialización y persistencia	15
3.10. Secciones	15
3.11. Referencias	15
3.12. Imágenes	15
3.13. Listas de items	16

3.14. Tablas	16
4. Técnicas y herramientas	19
5. Aspectos relevantes del desarrollo del proyecto	21
6. Trabajos relacionados	23
7. Conclusiones y Líneas de trabajo futuras	25
7.1. Conclusiones	25
7.2. Líneas de trabajo futuras	26
7.3. Conclusión final	28
Bibliografía	29

Índice de figuras

3.1. Autómata para una expresión vacía	16
--	----

Índice de tablas

3.1. Comparación de Scratch y Blockly	9
3.2. Comparación de LEGO WeDo 2.0 y LEGO SPIKE Prime . . .	10
3.3. Herramientas y tecnologías utilizadas en cada parte del proyecto	17

1. Introducción

En la última década, el interés por la enseñanza de la programación y la robótica ha crecido de manera significativa, con especial énfasis en la educación preuniversitaria. Esto se debe a la necesidad de dotar a los estudiantes de habilidades en pensamiento computacional, resolución de problemas y lógica algorítmica, consideradas esenciales en el mundo digital actual[8].

Entre las metodologías más empleadas en la enseñanza de la programación, la **programación basada en bloques** ha demostrado ser una herramienta eficaz, al facilitar la comprensión de la lógica de programación sin la complejidad de la sintaxis textual [4].

Actualmente, dos de las plataformas más destacadas en este ámbito son **Scratch y Blockly**.

Scratch, desarrollado por el MIT Media Lab, se ha consolidado como una de las herramientas educativas más utilizadas en la enseñanza de la programación a niños y jóvenes, proporcionando un entorno accesible y lúdico [9].

Por otro lado, Blockly, desarrollado por Google, ofrece una biblioteca de código abierto que permite integrar programación por bloques en diversas plataformas y convertir los bloques visuales a lenguajes de programación textuales como JavaScript y Python [4].

Aunque **Scratch se basa en Blockly**, existen diferencias fundamentales entre ambos sistemas, especialmente en términos de flexibilidad y aplicación. Scratch se centra en la educación infantil y no permite la exportación directa a código, mientras que Blockly está orientado a desarrolladores y educadores que requieren una mayor versatilidad en la generación de código [1].

La programación por bloques, utilizada en entornos como Scratch y Lego WeDo 2.0, ha sido ampliamente adoptada en educación debido a su accesibilidad y eficacia pedagógica [9]

Estudios previos han demostrado que herramientas como Scratch y Lego Mindstorms favorecen la enseñanza de fundamentos de programación y aumentan la tasa de aprobación en asignaturas de introducción a la programación [4]

Además, la gamificación ha emergido como un enfoque clave en la enseñanza de programación, promoviendo la motivación y el compromiso del estudiante[1]

No obstante, a pesar de estos avances, existen desafíos significativos para la enseñanza de la programación y robótica, especialmente en zonas con acceso limitado a recursos educativos y tecnológicos. La falta de laboratorios físicos y la dependencia de materiales costosos limitan la accesibilidad de estas herramientas a estudiantes en entornos rurales o con restricciones económicas [8]

Este problema está alineado con los Objetivos de Desarrollo Sostenible (ODS), en particular el ODS 4 (Educación de calidad) y el ODS 10 (Reducción de las desigualdades), que buscan garantizar el acceso equitativo a una educación inclusiva y de calidad [7]..

Frente a esta problemática, los entornos de simulación virtual han demostrado ser una solución eficaz para democratizar el acceso a la enseñanza de la programación y robótica, al permitir a los estudiantes experimentar sin necesidad de hardware físico [6]. Estudios previos han demostrado que el uso de juegos serios y simulaciones interactivas mejora la retención del conocimiento y la motivación en entornos educativos [5].

Dado que Blockly no está completamente adaptado a la estructura de Scratch, en este proyecto no solo se pretende evitar una simple integración, sino que se plantea el desarrollo de una **implementación híbrida** que combine las ventajas de ambos sistemas. Para ello, se analizará la implementación de **UBlockly**, una biblioteca de programación por bloques en Unity, que aunque no está diseñada específicamente para replicar los bloques de Scratch, puede servir como referencia para la adaptación del sistema propuesto en este trabajo [3].

Este proyecto también plantea un desafío técnico importante: la integración de un entorno de programación visual en un motor de simulación en 3D. Para ello, se explorarán estrategias utilizadas en otras plataformas como CoSpaces Edu, Robot Virtual Worlds, VEXcode VR y CoderZ [2] ,

identificando sus fortalezas y limitaciones para la creación de una solución mejorada. Asimismo, se analizará el impacto de la gamificación y la accesibilidad en el aprendizaje, con el objetivo de diseñar un entorno intuitivo y motivador que pueda ser utilizado por estudiantes con distintos niveles de experiencia.

2. Objetivos del proyecto

2.1. Introducción

El presente Trabajo Fin de Grado tiene como objetivo principal el desarrollo de un laboratorio virtual de programación y robótica que permita a los estudiantes aprender programación mediante bloques. Este laboratorio será desarrollado en el motor de videojuegos **Unity 6** y utilizando el lenguaje de programación **CSharp**.

La educación en robótica y programación ha demostrado ser una herramienta clave en el aprendizaje del pensamiento computacional. Sin embargo, uno de los principales desafíos en la enseñanza de la robótica es la dependencia de hardware físico, lo que limita el acceso a estudiantes en contextos de escasos recursos.

Para abordar esta problemática, el laboratorio virtual combinará un **entorno de simulación 3D**, que permitirá visualizar la ejecución de los programas, con un **sistema de bloques de programación**, basado en la lógica de **Scratch**, adaptado específicamente para la robótica educativa con **LEGO WeDo 2.0**.

Además, se analizará el repositorio **UBlockly**, una implementación en Unity que permite la programación por bloques. Aunque UBlockly no está diseñado específicamente para replicar los bloques de Scratch, representa una base sólida para evaluar su adaptabilidad e integración en este trabajo [3]. Su estudio permitirá definir estrategias para mejorar la implementación y optimización del sistema propuesto en este proyecto.

Para ello, se establecen los siguientes objetivos específicos:

2.2. Objetivos funcionales

Estos objetivos van a estar relacionados con las funcionalidades que el laboratorio virtual debe cumplir para ofrecer una experiencia educativa completa.

1. Diseño e implementación de un sistema de programación por bloques

- Replicar la lógica y los procedimientos de Scratch
- Analizar la implantación de UBlockly como referencia para la integración de bloques en Unity para el diseño de los siguientes bloques:
 - Bloques de movimiento, que permitan el desplazamiento del robot en diferentes direcciones.
 - Bloques de control, como bucles y condicionales para la toma de decisiones.
 - Bloques de sensores, para la introducción de valores para completar los bloques.
 - Bloques de operadores, que permitan realizar operaciones matemáticas y lógicas.
 - Bloques de eventos, que permitan ejecutar acciones en función de interacciones específicas tipo bucles.
 - Bloques de variables, que permitan almacenar y modificar valores.
 - Bloques de funciones, que permitan agrupar bloques y reutilizarlos.
- Evaluar qué aspectos de UBlockly pueden ser adaptados o mejorados para replicar la lógica de Scratch

2. Integración del sistema de bloques en Unity.

- Entorno de programación visual: interfaz interactiva donde se podrán arrastrar y soltar bloques uniéndolos igual que se realiza en Scratch.
- Sistema de ejecución de bloques: traducción de los bloques en acciones para que sean ejecutadas por la aplicación desarrollada en Unity

- Entorno de simulación en 3D: Visualización del comportamiento del robot en un entorno simulado donde se desplazará y ejecutará las acciones que se han programado con los bloques.
- Interfaz intuitiva: Orientada a la accesibilidad y facilidad de uso.

3. Adaptar el sistema de bloques al robot LEGO WEDO 2.0

- Crear bloques específicos para el control de los motores y sensores del robot.
- Implementar un sistema de comunicación entre el entorno de programación y el robot.
- Diseñar un entorno de simulación que permita visualizar el comportamiento del robot.
- Configurar bloques específicos para la utilización con el robot.

2.3. Objetivos no funcionales

Estos objetivos están enfocados en la accesibilidad, rendimiento y usabilidad del sistema.

1. Accesibilidad y escalabilidad

- Garantizar la accesibilidad del sistema a zonas con recursos limitados
- Diseñar una plataforma que pueda ejecutarse en distintos dispositivos sin requerimientos técnicos elevados.
- Permitir el acceso a contenidos educativos sin depender de equipamiento costoso.
- Explorar estrategias para que la herramienta pueda ser utilizada en contextos educativos con poca infraestructura tecnológica.

2. Investigación y análisis

- Investigar y analizar plataformas existentes como CoSpaces Edu, Robot Virtual Worlds, VEXcode VR y CoderZ.
- Identificar las ventajas y limitaciones de cada una y definir qué aspectos pueden ser mejorados en la solución propuesta.

3. Formación

- Adquirir conocimientos en Unity relacionados con la creación de entornos interactivos y simulaciones educativas.
- Aplicar buenas prácticas en la integración de sistemas de programación visual dentro del motor de simulación.

3. Conceptos teóricos

3.1. Paleta de colores

Para la implementación del laboratorio virtual en este Trabajo Fin de Grado, se han evaluado diversas plataformas de programación visual, comparando sus características clave.

Comparación entre Scratch y Blockly

Característica	Scratch	Blockly
Paleta de colores	Fija (según categorías)	Personalizable
Categorización de bloques	Movimiento, Control, Sensores, Eventos, Operadores, Variables, Funciones	Modular y personalizable
Exportación de código	No permite exportar código	Genera código en múltiples lenguajes (JavaScript, Python, Lua)
Flexibilidad	Cerrado, no permite modificar bloques	Altamente personalizable
Uso principal	Aprendizaje de programación visual en educación	Desarrollo de interfaces de programación visual para integración con código real

Tabla 3.1: Comparación de Scratch y Blockly

Comparación entre Lego Wedo 2.0 y Lego spike

Característica	LEGO WeDo 2.0	LEGO SPIKE Prime
Paleta de colores	Basada en íconos	Adaptación de Scratch, con bloques en formato palabra e íconos
Categorización de bloques	Basada en hardware (motores, sensores)	Similar a Scratch, con bloques específicos para hardware LEGO
Tipo de bloques	Bloques verticales con íconos sin palabras	Bloques de palabras en horizontal y algunos bloques con íconos
Exportación de código	No permite exportar código	No permite exportar código
Flexibilidad	Cerrado, bloques predefinidos	Más flexible que WeDo 2.0, pero limitado al ecosistema LEGO
Uso principal	Programación básica para principiantes en robótica educativa	Programación avanzada para proyectos de robótica con mayor capacidad de personalización

Tabla 3.2: Comparación de LEGO WeDo 2.0 y LEGO SPIKE Prime

LEGO SPIKE Prime es una adaptación de Scratch hecha por LEGO, por lo que mantiene una estructura de bloques similar a Scratch, pero ajustada a su ecosistema de hardware.

LEGO WeDo 2.0 usa una interfaz más sencilla con bloques puramente icónicos (sin palabras), mientras que SPIKE Prime combina texto e íconos.

3.2. Implementación de UBlockly en Unity

UBlockly es una librería basada en Blockly que permite la integración de bloques de programación en Unity. Si bien no está diseñada específicamente para replicar Scratch, su estructura modular y personalizable puede servir como base para desarrollar un entorno de programación similar.

Arquitectura de uBlockly

La arquitectura de uBlockly se divide en tres capas principales:

- **Modelo Core:** Gestiona la estructura lógica de los bloques, incluyendo el espacio de trabajo **Workspace**, la base de datos de bloques y las conexiones entre ellos.
- **Interfaz de usuario (UI):** Maneja la representación visual de los bloques y su interacción dentro del editor de Unity.
- **Ejecución de código:** Se encarga de interpretar y ejecutar la lógica definida en los bloques.

Espacio de trabajo / Workspace

El Workspace es el contenedor principal donde se almacenan y organizan los bloques. Su principal función es gestionar la disposición y persistencia de los bloques dentro del entorno de programación.

- Atributos principales del Workspace
 - **Id:** Identificador único del espacio de trabajo.
 - **Options:** Configuraciones del tipo modo de solo lectura, ejecución sincrónica y dirección del texto.
 - **TopBlocks:** Lista de bloques que no están conectados a otros.
 - **BlockDB:** Diccionario con todos los bloques almacenados por su ID
 - **VariableMap:** Estructura para el manejo de variables en el workspace.
 - **ConnectionList:** Base de datos de conexiones entre bloques.
 - **ProcedureDB:** Base de datos de procedimientos definidos dentro del espacio de trabajo.

Creación y eliminación de bloques

El espacio de trabajo permite la creación de nuevos bloques mediante el uso de la clase BlockFactory y otros métodos para la eliminación y gestión de bloques existentes.

3.3. Bloques

Cada bloque representa una unidad funcional dentro del Workspace y pueden contener los siguientes elementos:

- **Connection:** Conexiones de entrada y salida
- **VariableMap:** Relación de variables
- **Field:** Campos de texto o de selección
- **Input:** Entradas anidadas
- **ChildBlocks:** Bloques hijos

Jerarquía de un bloque

Cada bloque puede estar compuesto por diferentes elementos jerárquicos:

- **Block:** Bloque superior en el espacio de trabajo
 - **ConnectionOutput** Conexión de salida
 - **ConectionPrev** Conexión previa
 - **ConnectionNext:** Conecta un bloque con el siguiente dentro de la secuencia de ejecución Block(Next)
- **Input:** Este puede ser de varios tipos:
 - **Field:**
 - **ConnectonInput:** Conexión de entrada

3.4. Conexiones

Los bloques pueden unirse entre sí a través de un sistema de conexiones. Cada bloque puede tener tres tipos de conexiones principales:

- **outputConnection:** Conecta la salida de un bloque con la entrada de otro
- **NextConnection:** Conecta un bloque con su bloque superior
- **PreviousConnection:** Enlaza un bloque con su bloque superior

Implementación de Connection en uBlockly

Clase Connection

- Verifica la compatibilidad entre conexiones y proporciona funciones para conectar y desconectar bloques mediante código.
- Determina que bloques son compatibles
- Maneja eventos como conexiones, desconexiones y reorganización de bloques

Atributos principales

- **SourceBlock:** Bloque de origen de la conexión
- **TargetConnection:** Conexión a la que se encuentra conectado el bloque
- **IsConnected** Indica si la conexión está activa
- **Check:** Lista de tipos de valores compatibles con la conexión
- **Location:** Posición de la conexión en coordenadas del tipo Vector2<int>

3.5. Entradas o Inputs

Un Input representa una entrada dentro de un bloque, manejando la alineación y organización de los diferentes elementos visuales dentro del bloque y entre sus atributos principales están los siguientes:

- **Connection:** La conexión que permite enlazar otro bloque dentro del actual
- **FieldRow:** Lista de Fields que almacenan valores o etiquetas dentro del bloque.
- **SourceBlock:** Bloque padre al que pertenece la entrada.
- **ConnectedBlock:** Bloque que esta conectado a esta entrada, si es que existe esta.
- **Input:** Son campos de datos que representan valores editables en el bloque

3.6. Campos o Fields

Los Field datos dentro de un bloque entre los que se encontrarían etiquetas de texto, entradas editables o selecciones. Sus atributos principales son los siguientes

- **Name:** Identificador único del campo dentro de un bloque
- **SourceBlock:** Bloque al que se encuentra asociado el campo
- **Validator:** Incluye funciones de validación para limitar los valores que se pueden ingresar.
- **IsImage:** Indica si este campo contiene una image en lugar de un texto.
- **PrefixField o SuffixField:** Son campos adicionales que nos indican que puede acompañar al campo principal dentro de un bloque.

3.7. Variables o VariableMap

El variableMap gestionar la creación, eliminación y renombrado de variables dentro del workSpace y entre sus atributos claves están los siguientes:

- **mVariableMap:** Diccionario de variables que esta agrupado por su tipo
- **mWorkSpace:** Hace referencia al espacio de trabajo asociado.

3.8. Representación de bloques en Unity

Para adaptar uBlockly a Unity, es necesario traducir cada bloque en un objeto GameObject con elementos UI como:

- **Image:** Representa la base del bloque.
- **TextMeshPro:** Representan las etiquetas de tipo texto.
- **Button:** Representan los botones para las interacciones.
- **Anchor Points:** Se utilizan para definir donde se realizan las conexiones entre bloques

- **LineRenderer:** Representación visual de las conexiones
- **Bezier Curves:** Refleja la relación entre los bloques en la UI

3.9. Serialización y persistencia

Para guardar y cargar bloques utiliza archivos en formato XML y JSON que nos permite almacenar la estructura del Workspace y restaurarla cuando sea necesario.

3.10. Secciones

Notas del proyecto preconfigurados En aquellos proyectos que necesiten para su comprensión y desarrollo de unos conceptos teóricos de una determinada materia o de un determinado dominio de conocimiento, debe existir un apartado que sintetice dichos conceptos.

Algunos conceptos teóricos de L^AT_EX¹.

Las secciones se incluyen con el comando section.

Subsecciones

Además de secciones tenemos subsecciones.

Subsubsecciones

Y subsecciones.

3.11. Referencias

Las referencias se incluyen en el texto usando cite [?]. Para citar webs, artículos o libros [?], si se desean citar más de uno en el mismo lugar [?, ?].

3.12. Imágenes

Se pueden incluir imágenes con los comandos standard de L^AT_EX, pero esta plantilla dispone de comandos propios como por ejemplo el siguiente:

¹Créditos a los proyectos de Álvaro López Cantero: Configurador de Presupuestos y Roberto Izquierdo Amo: PLQuiz



Figura 3.1: Autómata para una expresión vacía

3.13. Listas de items

Existen tres posibilidades:

- primer item.
- segundo item.

1. primer item.
2. segundo item.

Primer item más información sobre el primer item.

Segundo item más información sobre el segundo item.

■

3.14. Tablas

Igualmente se pueden usar los comandos específicos de $\text{\LaTeX}$ o bien usar alguno de los comandos de la plantilla.

Herramientas	App	AngularJS	API REST	BD	Memoria
HTML5		X			
CSS3		X			
BOOTSTRAP		X			
JavaScript		X			
AngularJS		X			
Bower		X			
PHP			X		
Karma + Jasmine		X			
Slim framework			X		
Idiorm			X		
Composer			X		
JSON		X	X		
PhpStorm		X	X		
MySQL				X	
PhpMyAdmin				X	
Git + BitBucket		X	X	X	X
MikT _E X					X
T _E XMaker					X
Astah					X
Balsamiq Mockups		X			
VersionOne		X	X	X	X

Tabla 3.3: Herramientas y tecnologías utilizadas en cada parte del proyecto

4. Técnicas y herramientas

Esta parte de la memoria tiene como objetivo presentar las técnicas metodológicas y las herramientas de desarrollo que se han utilizado para llevar a cabo el proyecto. Si se han estudiado diferentes alternativas de metodologías, herramientas, bibliotecas se puede hacer un resumen de los aspectos más destacados de cada alternativa, incluyendo comparativas entre las distintas opciones y una justificación de las elecciones realizadas. No se pretende que este apartado se convierta en un capítulo de un libro dedicado a cada una de las alternativas, sino comentar los aspectos más destacados de cada opción, con un repaso somero a los fundamentos esenciales y referencias bibliográficas para que el lector pueda ampliar su conocimiento sobre el tema.

Conceptos Técnicos:

Entorno de desarrollo: Implementación del laboratorio virtual mediante Unity, integrando prefabs (bloques y piezas lego) y simulaciones en tiempo real Modelado 3D: Diseño y exportación de componentes robóticos con Blender para integrarlos en Unity, teniendo en cuenta la optimización de texturas y mallas. Programación orientada a bloques: Recreación de un motor de programación visual basado en Blockly/Scratch, integrando funcionalidades específicas para la simulación y control de robots. Control de versiones: Integración de Plastic SCM para gestión de cambios y sincronización con repositorios en Unity Cloud y GitHub, asegurando la trazabilidad del desarrollo. >Bases de datos en tiempo real: Uso de Firebase para gestionar y sincronizar datos entre usuarios, permitiendo almacenar información sobre proyectos, configuraciones y resultados en tiempo real. Diseño y modelado UML: Creación de diagramas UML para representar las interacciones del sistema, el flujo de datos y los componentes clave del

software. Compatibilidad con dispositivos VR: Diseño del entorno 3D para garantizar una experiencia fluida en gafas de RV como Oculus, considerando el rendimiento y la optimización para hardware específico. Validación y pruebas de software: Metodologías para garantizar la calidad del software, incluyendo pruebas unitarias para validar módulos individuales, pruebas de integración para garantizar la interoperabilidad de los componentes, y pruebas de usabilidad para evaluar la experiencia del usuario objetivo.

Aplicación de conceptos relacionados con metodologías ágiles.

5. Aspectos relevantes del desarrollo del proyecto

Este apartado pretende recoger los aspectos más interesantes del desarrollo del proyecto, comentados por los autores del mismo. Debe incluir desde la exposición del ciclo de vida utilizado, hasta los detalles de mayor relevancia de las fases de análisis, diseño e implementación. Se busca que no sea una mera operación de copiar y pegar diagramas y extractos del código fuente, sino que realmente se justifiquen los caminos de solución que se han tomado, especialmente aquellos que no sean triviales. Puede ser el lugar más adecuado para documentar los aspectos más interesantes del diseño y de la implementación, con un mayor hincapié en aspectos tales como el tipo de arquitectura elegido, los índices de las tablas de la base de datos, normalización y desnormalización, distribución en ficheros³, reglas de negocio dentro de las bases de datos (EDVHV GH GDWRV DFWLYDV), aspectos de desarrollo relacionados con el WWW... Este apartado, debe convertirse en el resumen de la experiencia práctica del proyecto, y por sí mismo justifica que la memoria se convierta en un documento útil, fuente de referencia para los autores, los tutores y futuros alumnos.

6. Trabajos relacionados

Este apartado sería parecido a un estado del arte de una tesis o tesina. En un trabajo final grado no parece obligada su presencia, aunque se puede dejar a juicio del tutor el incluir un pequeño resumen comentado de los trabajos y proyectos ya realizados en el campo del proyecto en curso.

7. Conclusiones y Líneas de trabajo futuras

Todo proyecto debe incluir las conclusiones que se derivan de su desarrollo. Éstas pueden ser de diferente índole, dependiendo de la tipología del proyecto, pero normalmente van a estar presentes un conjunto de conclusiones relacionadas con los resultados del proyecto y un conjunto de conclusiones técnicas. Además, resulta muy útil realizar un informe crítico indicando cómo se puede mejorar el proyecto, o cómo se puede continuar trabajando en la línea del proyecto realizado.

7.1. Conclusiones

El presente Trabajo Fin de Grado ha abordado el desarrollo de un **laboratorio virtual de programación y robótica** basado en el motor de videojuegos Unity y el lenguaje de programación C#. A lo largo del proyecto, se han implementado un **entorno de programación visual**, bloques de código inspirados en **Scratch**, y una simulación 3D para el robot LEGO WeDo 2.0.

Los principales logros del proyecto incluyen:

- Implementación de un sistema de bloques de programación con estructuras básicas como movimiento, control, operadores y eventos.
- Integración de estos bloques dentro de Unity, permitiendo su ejecución dentro del laboratorio.

- Desarrollo de una simulación 3D en Unity para visualizar la ejecución de los bloques en el robot virtual.
- Creación de una interfaz interactiva que facilita la programación por bloques.

Pese a los avances logrados, el proyecto presenta varias limitaciones, principalmente en lo referente a la validación con usuarios y la integración avanzada con hardware físico. Estas limitaciones abren la posibilidad de futuras líneas de investigación y desarrollo.

7.2. Líneas de trabajo futuras

El presente proyecto establece una base sobre la que se pueden desarrollar nuevas mejoras y ampliaciones. A continuación, se presentan algunas de las posibles líneas de trabajo futuras:

Extensión del sistema de bloques

Inicialmente, el sistema implementa bloques básicos que permiten la programación elemental del robot. En futuras versiones del laboratorio virtual, se pueden desarrollar:

- **Bloques avanzados:** Incorporación de estructuras más complejas como procedimientos, listas y operadores booleanos.
- **Expansión del modelo de eventos:** Implementación de eventos personalizados que permitan interacciones más complejas.
- **Sistema de depuración visual:** Desarrollo de herramientas que permitan a los estudiantes visualizar el flujo de ejecución de los bloques y detectar errores en tiempo real.

Mejora de la simulación 3D

En la aplicación creada, la simulación se centra en la ejecución de bloques. Sin embargo, para mejorar la experiencia de aprendizaje, se pueden introducir mejoras como:

- **Interacción con entornos dinámicos:** Incorporación de escenarios donde el robot pueda responder a estímulos del entorno.

- **Simulación de física avanzada:** Implementación de físicas más realistas que afecten la movilidad del robot dentro del entorno 3D.
- **Soporte para distintos modelos de robots:** Expansión del sistema para incluir otros robots educativos más allá del LEGO WeDo 2.0 como pueden ser Lego spike Essential, Lego Spike Prime y Lego EV3 MindStorm.

Evaluación de la efectividad educativa

Una de las principales limitaciones del presente trabajo es la falta de validación con usuarios reales. Para futuras adaptaciones, se propone:

- **Estudios de usabilidad:** Realización de pruebas con estudiantes para evaluar la facilidad de uso de la interfaz.
- **Análisis del impacto en el aprendizaje:** Medición sobre cómo la plataforma mejora la enseñanza de programación en comparación con otros métodos.
- **Recopilación de métricas de interacción:** Implementación de herramientas de análisis para evaluar cómo los estudiantes utilizan el sistema.

Integración con hardware físico

Si bien el sistema está diseñado para replicar la programación del LEGO WeDo 2.0, la integración directa con hardware real no ha sido abordada en este trabajo por lo que algunas mejoras en esta línea pueden incluir:

- **Conexión en tiempo real con LEGO WeDo 2.0:** Desarrollo de una API que permita la ejecución de código directamente en el robot físico mediante tecnología Bluetooth.
- **Compatibilidad con otros kits de robótica:** Ampliación del sistema para permitir la programación de otros dispositivos como Arduino o micro:bit.

Optimización del rendimiento y accesibilidad

Otra futura línea de trabajo, que se abre, es la optimización del sistema para hacerlo más accesible:

- **Optimización de Unity:** Reducción del consumo de recursos para permitir su ejecución en dispositivos de bajo rendimiento.
- **Versión web del entorno:** Desarrollo de una versión ejecutable en navegadores sin necesidad de instalación.
- **Localización del sistema:** Traducción del entorno a múltiples idiomas para expandir su alcance educativo.

7.3. Conclusión final

(pendiente)El desarrollo de este laboratorio virtual representa un avance en la enseñanza de programación mediante entornos visuales. Con futuras mejoras, el sistema podría convertirse en una herramienta educativa más robusta, accesible y efectiva para la formación en robótica y pensamiento computacional.

Bibliografía

- [1] Jesús R. Campaña, Ana E. Marín, María Ros, Daniel Sánchez, Juan M. Medina, María Amparo Vila, María Dolores Ruiz, Manuel P. Cuéllar, and María J. Martín-Bautista. Metodologías activas y gamificación en las asignaturas de iniciación a la programación. In *Actas de las JENUI - Vol. 1 (2016)*, Almería, jul 2016. [En línea; consultado el 12-febrero-2025].
- [2] Rafael Arnay Coromoto León Cristian Manuel Ángel Diaz, Eduardo Segredo. Simulador de robótica educativa para la promoción del pensamiento computacional. 2020. [En línea; consultado el 28-enero-2025].
- [3] Imagicbell. UBlockly - Introducción a la Programación con Bloques en Unity, 2021. [En línea; consultado el 20-febrero-2025].
- [4] Roberto Muñoz, Thiago S. Barcelos, Rodolfo Villarroel, Marta Barría, Carlos Becerra, Rene Noel, and Ismar Frango Silveira. Uso de scratch y lego mindstorms como apoyo a la docencia en fundamentos de programación. In *Actas de las XXI Jornadas de Enseñanza Universitaria de la Informática, JENUI 2015*, Andorra La Vella, jul 2015. [En línea; consultado el 12-febrero-2025].
- [5] Héctor Pérez, José L. García, and Marta Fernández. Simulador 3d de robótica distribuido en unity para enseñanza de programación e inteligencia artificial. *ResearchGate*, 2024. [En línea; consultado el 20-febrero-2025].
- [6] John Smith and Jane Doe. Geobotsvr: A robotics learning game for beginners with hands-on learning simulation. *arXiv*, 2024. [En línea; consultado el 20-febrero-2025].

- [7] Naciones Unidas. Objetivos de desarrollo sostenible, 2023. [En línea; consultado el 20-febrero-2025].
- [8] Maximiliano Paredes Velasco and J. Ángel Velázquez-Iturbide. Una asignatura para la formación del profesorado en programación mediante lenguajes basados en bloques. In *Actas de las JENUI - Vol. 7 (2022)*, pages 343–350, La Coruña, jul 2022. [En línea; consultado el 11-febrero-2025].
- [9] Maximiliano Paredes Velasco, Jesús Ángel Velázquez Iturbide, Sergio Cervero Díaz, and Daniel Palacios Alonso. Mejora de una asignatura para la formación del profesorado en programación basada en bloques. In *Actas de las JENUI - Vol. 8 (2023)*, pages 269–276, Granada, jul 2023. [En línea; consultado el 11-febrero-2025].